Reviewer Pack — Minimal Local Cohomology in Algebraic Geometry of Boolean Fu…

Entry 431db2d625af · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 03:15:50 UTC

Minimal Local Cohomology in Algebraic Geometry of Boolean Functions and Communication Complexity Rank Variance

Entry ID: `431db2d625af` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 03:15:50 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Boolean Functions **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every n-vertex d-regular graph G, the minimal local cohomology group H^1(G) with respect to its ideal I_G induced by the algebraic geometry of boolean functions is linearly correlated with its communication complexity rank variance R_var(G), such that log|H^1(G)| = Θ(R_var(G)).

Rationale (proposer's reasoning):

> The algebraic geometry of boolean functions provides a rich structure for analyzing Boolean circuits, and local cohomology groups capture the obstruction to certain geometric invariants. The communication complexity rank variance quantifies the difficulty of distributing information among two parties. This conjecture suggests that the geometric structure of Boolean functions can shed light on the distributional complexities of communication tasks.

Taxonomy category: `geometric_algebraic_structure` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fddec5dfd6733118`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For n-vertex d-regular graphs, if the Pearson correlation coefficient between log|H^1(G)| and R_var(G) is ≥ 0.8 across all seeds, then support for the conjecture is provided. If any seed yields a correlation coefficient < 0.8 or a p-value > 0.05, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Cohomology Algebraic Geometry Boolean Functions AND Communication Complexity Rank Variance - H^1(G) Ideal I_G Boolean Functions Correlation Communication Complexity Rank Variance - Boolean Function Algebraic Geometry H^1(G) Ideal Correlation Matrix Rank Variance

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random n-vertex d-regular graph
    n = 20
    d = 3
    if n % d != 0:
        return {
            "metric_name": "log|H^1(G)| vs R_var(G)",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "Graph size must be a multiple of the degree"
        }

    G = {}
    for i in range(n):
        neighbors = random.sample([j for j in range(n) if j != i], d)
        G[i] = set(neighbors)

    # Compute the associated algebraic geometry of boolean functions
    # and determine the ideal I_G (simplified for this example)
    I_G = set()
    for node in G:
```

```
        for neighbor in G[node]:
            I_G.add((node, neighbor))

    # Calculate the minimal local cohomology group H^1(G) (simplified)
    H_1_G = len(I_G)

    # Compute the communication complexity rank variance R_var(G)
    # (simplified for this example)
    R_var_G = H_1_G / n

    # Correlate log|H^1(G)| with R_var(G) using a statistical test
    log_H_1_G = math.log(H_1_G) if H_1_G > 0 else -math.inf
    correlation_coefficient = (log_H_1_G * R_var_G) / (n * n)

    return {
        "metric_name": "log|H^1(G)| vs R_var(G)",
        "metric_value": correlation_coefficient,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(correlation_coefficient) >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = "N/A"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'conjecture_ho
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
```

```
TRIAL: {'metric_name': 'log|H^1(G)| vs R_var(G)', 'metric_value': None, 'instances_tested': 0, 'n_max
RESULT: FALSIFIED counterexamp
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code is not calculating the minimal local cohomology group H^1(G) correctly, as it simply counts the number of elements in the ideal I_G. The actual computation of H^1(G) requires a more sophisticated algebraic algorithm that is not implemented here.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test code failed to compute the minimal local cohomology group H^1(G) correctly and produced counterexamples that do not meet the pre-registered s | next: Implement a correct computation of the minimal local cohomology group H^1(G) and retest the conjecture to ensure it meets the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14473
2	propose	ollama_remote	glm4:latest	0	0	13105
3	propose	ollama_remote	glm4:latest	0	0	13505
4	preregistration	ollama_remote	glm4:latest	0	0	12406
5	novelty	ollama_remote	glm4:latest	0	0	11591
6	novelty	ollama_remote	glm4:latest	0	0	15465
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44620
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14192
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11094
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9439
11	critic	ollama_remote	glm4:latest	0	0	20379
12	judge	ollama_remote	glm4:latest	0	0	9508

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 189777 ms total latency. Provider mix: {‘ollama_remote’: 12}

(full prompt+response transcripts available in `research/audit/431db2d625af.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/431db2d625af.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/431db2d625af.tar.gz` (if generated)